

cxor

A One-Bit Context Transform over $\mathbb{F}_2$ and Its Sixteen Update Rules

Yauhen Yakimovich
yauhen.yakimovich.mail@gmail.com

August 20, 2026

Abstract

Context Transformation Algorithms (CTA) originated as a broader family of context-addressed transforms. This paper isolates one binary member, called cxor (CTA-XOR), in which every m -bit context addresses a single bit of state and the source symbol is combined with that state by addition in $\mathbb{F}_2$. Reading the state as a hard next-bit prediction makes the emitted bit a prediction residual: zero denotes a correct prediction and one an error. The historically implemented update $W_5(s, x) = x$ replaces the prediction with the observation, so a context predicts that its next successor will equal its most recently observed successor.

cxor is not itself a compressor: it is length preserving and bijective. Its possible use is to expose contextual predictability as a zero-rich residual for a subsequent coder. We prove that sequential decoding follows from causality and does not depend on the particular update rule. Under the deliberately narrow one-bit cxor assumptions, an update is an arbitrary Boolean function of the old state and current symbol, giving exactly sixteen wirings. Algebraic normal form separates eight affine rules, whose per-context behaviour reduces to the three operators 1 , $1 + T$, and $(1 + T)^{-1}$, from eight data-dependent rules. The same analysis yields four adaptation horizons. W_5 realises contextual differencing and forgets an old prediction after one observation; W_6 realises running parity and retains its old state indefinitely. At context order zero these are the two directions of binary Gray coding.

1 From context modelling to cxor

Statistical compression begins by assigning probabilities to the next symbol. In a binary order- m context model, the m most recent source bits select one of 2^m contexts, and the model records what has followed that context in the past. Prediction by partial matching (PPM), for example, keeps symbol counts at several context orders and uses an escape mechanism when a long context is uninformative [BCW90]. A probability model and an entropy coder then turn accurate probability estimates into a short representation.

Context Transformation Algorithms (CTA) denotes the broader idea of associating state with contexts and using it to transform a source. This paper studies one deliberately restricted binary member, which we call cxor (CTA-XOR):

Context Transformation Algorithms (CTA) $\supset$ cxor over $\mathbb{F}_2 \supset$ the historical W_5 instance.

cxor can be reached from context modelling by asking how little of the architecture can remain. Give each context exactly one bit of state and interpret that bit as a hard

prediction $\hat{x}$ of the next source bit. When the actual bit x arrives, emit

$$r = x \oplus \hat{x}. \quad (1)$$

Then $r = 0$ means that the context predicted correctly, while $r = 1$ means a prediction error. The result is simultaneously a context-indexed predictor and a residual transform.

The historically implemented update is as small as the state. After observing x , store x in the cell for the current context:

$$W_5(s, x) = x. \quad (2)$$

Thus the next time the same context occurs, W_5 predicts that it will have the same successor as last time. It has one bit per context, immediate adaptation, and no counts, probabilities, confidence, escape, or backoff. This predictor interpretation is not an analogy added after the fact: it is exactly the read–compare–update sequence in the earlier cxor implementation.

The residual can be more skewed than the source. With context order $m = 4$, the example shipped with the reference W_5 program is

```
input: 10100111011101111101111111111110
cxorW5: 101001110111000011100001000000001
inverse: 10100111011101111101111111111110.
```

The input has 7 zeros and 26 ones; its image has 20 zeros and 13 ones. The useful observation is zero enrichment: repeated contextual regularity has become a concentration of prediction successes. The example is not evidence of compression, and no compression result is claimed here. cxor emits exactly one bit for every input bit and can be inverted exactly.

The paper first defines this predictor and its decoder, then makes precise which regularities W_5 can expose and why their exposure is separate from both invertibility and compression. Only after that do we vary the learning rule. Within the one-bit cxor architecture, if a cell may be updated using only its old value s and the observed bit x , the update is a Boolean function

$$w : \mathbb{F}_2^2 \longrightarrow \mathbb{F}_2,$$

and there are exactly $2^{2^2} = 16$ possibilities. This finite design space classifies the narrow cxor model, not the CTA family.

The construction studied here originated in earlier unpublished work by the author on the broader CTA family. That work existed in manuscript and implementation form, but the binary XOR construction was never published as a separate, self-contained treatment. The present paper extracts that operation from the broader line of work, formalises it independently, and calls it cxor.

The analysis makes four contributions.

1. It identifies the historical W_5 implementation as a last-observation context predictor followed by an exclusive-or residual.
2. It proves that causal prediction makes every one of the sixteen transforms sequentially invertible; reversibility comes from the predictor/residual construction, not from a special feature of W_5 .

3. It classifies the rules by algebraic normal form. The eight affine rules admit a compact per-context operator description and realise only 1 , $1 + T$, or $(1 + T)^{-1}$; the other eight have data-dependent state retention.
4. It interprets the resulting memory horizons as adaptation behaviour. W_5 replaces an old belief after one observation, whereas $W_6(s, x) = s \oplus x$ preserves old state.

Three notions will remain separate throughout. *Prediction performance* is a property of a rule, a context order, and a particular source; it can be measured by the residual zeros. *Invertibility* is structural and holds even when every prediction is poor. *Compression* requires an additional model and coder. Confusing any two of these obscures what cxor does.

2 The cxor architecture

Let $\mathbb{F}_2 = \{0, 1\}$, with addition interpreted as exclusive-or. Fix a context order $m \geq 0$. The context set is $A = \mathbb{F}_2^m$, and a table $s : A \rightarrow \mathbb{F}_2$ contains one bit for every context. Before the input starts, the table has a known initial value s_0 and the context window has a known prefix $\pi \in \mathbb{F}_2^m$. The reference W_5 program uses zeros for both.

For an input $x = x_0x_1 \dots x_{n-1}$, the address a_k at position k is the length- m suffix of $\pi x_0 \dots x_{k-1}$. In particular, a_k contains only source symbols *strictly before* x_k . At a step, write $u = s[a_k]$ for the state read from the addressed cell. The general datapath is

$$r_k = x_k + u, \quad s[a_k] \leftarrow w(u, x_k), \quad (3)$$

where w is a one-bit update rule. The historical instance uses $w = W_5$, so the second assignment is simply $s[a_k] \leftarrow x_k$.

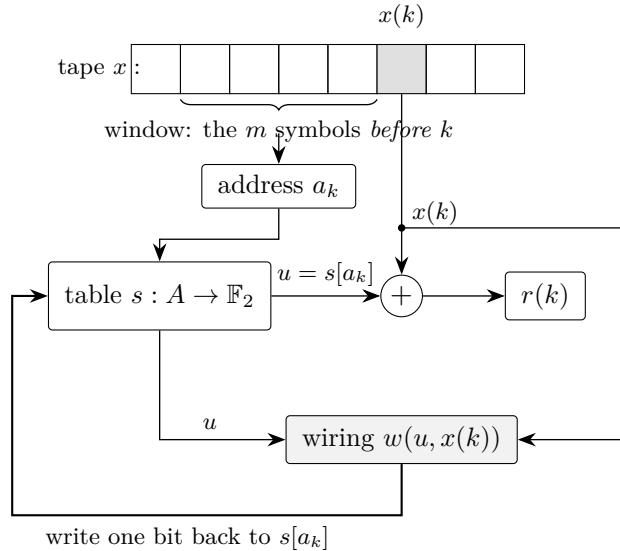


Figure 1: One cxor step. The preceding m source symbols address a one-bit prediction; the exclusive-or emits a residual; and the update rule writes the addressed cell. In the historical W_5 implementation the shaded update is $W_5(u, x) = x$.

The ordering matters. The lookup precedes the window shift, so the prediction is causal. The current symbol is used to update the prediction only after its residual has been emitted.

Algorithm 1: cxor_w ; choose $w = W_5$ for the historical instance

```

1  $c \leftarrow \pi; s \leftarrow s_0$ 
2 for  $k \leftarrow 0$  to  $n - 1$  do
3    $a \leftarrow c; u \leftarrow s[a]$ 
4    $r_k \leftarrow x_k + u$ 
5    $s[a] \leftarrow w(u, x_k)$ 
6    $c \leftarrow \text{suffix}_m(cx_k)$ 
7 return  $r$ 

```

2.1 Why the reference code is exactly W_5

The reference implementation expresses its update conditionally:

```

diff = state != x
out_bits.append(diff)
if diff:
    lookup[index] = x

```

For bits, `state != x` is $s[a_k] \oplus x_k$, so `diff` is precisely the residual r_k . When `diff` is true, the code stores x_k . When it is false, the assignment is unnecessary because $s[a_k] = x_k$ already. Consequently the conditional is equivalent to the unconditional update

$$s[a_k] \leftarrow x_k,$$

which is W_5 . Operationally, the program reads the prediction, reports whether it was wrong, and makes the new prediction equal to the observation.

2.2 A small trace

Take $m = 2$, a zero prefix and table, and the periodic input $x = 010101010$. The first occurrences of contexts 01 and 10 establish their successors. Thereafter the same associations repeat, so their residuals are zero.

k	context a_k	source x_k	prediction u	residual r_k	new prediction
0	00	0	0	0	0
1	00	1	0	1	1
2	01	0	0	0	0
3	10	1	0	1	1
4	01	0	0	0	0
5	10	1	1	0	1
6	01	0	0	0	0
7	10	1	1	0	1
8	01	0	0	0	0

Table 1: W_5 on a two-bit context. The output is 010100000. Each zero is a correct hard prediction, including initial predictions from the zero table.

The trace also shows why periodicity is only a special case of the mechanism. The machine does not search for a period; it independently remembers that 01 was followed by 0 and 10 by 1.

2.3 Sequential decoder

The residual loses no information. A decoder starts with the same prefix, table, and update rule. Suppose it has already recovered $x_0, \dots, x_{k-1}$. Those bits determine a_k , and replaying the same updates determines the encoder's current $u = s[a_k]$. Hence

$$x_k = r_k + u. \quad (4)$$

The decoder then applies $s[a_k] \leftarrow w(u, x_k)$ and advances the window with the recovered symbol. This proves the following general fact.

Theorem 2.1 (Causal residuals are invertible). *For every m , initial table s_0 , prefix π , deterministic update rule w , and length n , Algorithm 1 is a bijection $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$.*

Proof. Equation (4) recovers the input sequentially. Induction on k shows that encoder and decoder have identical windows and tables before every step, so the recovery is unique. The resulting inverse maps the finite set $\mathbb{F}_2^n$ onto itself. $\square$

The proof does not invert w and uses no special property of W_5 . It inverts the exclusive-or emitter using a prediction reproducible from the already decoded prefix. This is the same causal principle used by predictive lossless codes with much richer predictors.

If the window were shifted before lookup, so that x_k participated in its own address, the decoder would in general need x_k to discover the prediction required to recover x_k . Collisions then occur for ordinary nonzero tables. The lagging, source-driven address in Algorithm 1 is therefore part of the transform, not a cosmetic indexing convention.

3 The W_5 instance as a predictive transform

The one-bit state has a direct operational meaning under W_5 . Fix a context a , and list the positions at which it occurs as

$$k_0 < k_1 < k_2 < \dots$$

Let $y_j = x_{k_j}$ be the symbol that follows that occurrence of a . If the initial cell is q_a , exor_{W_5} emits

$$r_{k_0} = y_0 + q_a, \quad r_{k_j} = y_j + y_{j-1} \quad (j \geq 1). \quad (5)$$

The first visit is predicted from the initial table. Every later visit compares the current successor with the successor observed on the preceding visit to the same context. Thus W_5 is contextual differencing: after the first visit, a zero says exactly that the association $a \mapsto y$ has repeated.

This gives a more accurate description than saying that W_5 detects periodic substrings. W_5 suppresses repeated context-to-successor associations. Periodic and locally repetitive sources are important special cases: once an m -bit context recurs with a stable successor, subsequent visits to that context emit zero. But periodicity is neither necessary nor sufficient as a source-wide description. Different contexts may recur at irregular intervals, and a repeated context whose successor alternates will produce errors rather than zeros.

3.1 What determines zero enrichment

The residual depends jointly on the source and the model order.

- *Context order.* A small m merges histories that may have different successors; a large m separates them but creates more cells to learn.
- *Input length and recurrence.* There are 2^m possible contexts. On a short input, many are never revisited, so their one-bit memories cannot contribute a learned prediction. A context seen only once is predicted solely from its initial cell.
- *Successor stability.* Recurrence helps only when repeated occurrences tend to have the same following bit. Equation (5) makes this dependence exact.
- *Initialisation.* The initial table affects the first visit to each context. The zero table used by the reference implementation initially predicts zero everywhere; W_5 overwrites that choice after one observation.

The example in the introduction illustrates these conditions rather than proving a general gain. Its 33 symbols are long enough to revisit several four-bit contexts, and enough of those contexts repeat their successor that the residual contains 20 zeros instead of 7. On another source, or with another m , W_5 may leave the marginal balance unchanged or make it less favourable. Prediction performance is therefore empirical even though the transform is exact.

3.2 Why this is not compression

cxor_w maps n input bits to n residual bits, and Theorem 2.1 says that it permutes the 2^n possible strings for every wiring. It does not reduce length or information entropy by itself. The intended architecture, if the residual is useful, is

$$x \longrightarrow \text{cxor}_{W_5} \text{ residual} \longrightarrow \text{statistical model and entropy coder}.$$

A downstream coder might exploit a residual with a strong zero bias, long zero runs, or other simple structure more easily than it exploits the original context dependence. Whether it actually does so depends on that coder and on the source; it is an experimental question not answered by the algebra in this paper.

The role is analogous in spirit to other reversible preprocessing transforms. The Burrows–Wheeler transform, for example, is valuable because it presents existing dependence in a form suitable for simple back ends, not because the permutation itself shortens the data [BW94]. cxor_{W_5} uses a causal predictor rather than block sorting, so the analogy should not be taken further.

Zero frequency, invertibility, and compressed size therefore answer different questions. The first measures hard-prediction success for a specific run. The second is guaranteed by causal decoding. The third can be known only after a downstream representation has been specified.

4 The sixteen one-bit update rules

The one-bit cxor model fixes the address calculation and residual emitter, leaving only the cell update to vary. If the update can inspect only the old state s and current source bit x , it is a function $w : \mathbb{F}_2^2 \rightarrow \mathbb{F}_2$. Four input pairs must each be assigned one output bit, so there are

$$2^{|\mathbb{F}_2^2|} = 2^4 = 16$$

rules. This is the complete update space under the one-cell, one-bit cxor restriction.

We name a rule by reading

$$w(0,0) w(0,1) w(1,0) w(1,1)$$

as a four-bit binary integer, most significant bit first. Thus the projection $w(s, x) = x$ has truth table 0101 and is W_5 , while exclusive-or $w(s, x) = s + x$ has table 0110 and is W_6 .

Every two-variable Boolean function has a unique algebraic normal form (ANF)

$$w(s, x) = d_0 + d_1 s + d_2 x + d_3 s x, \quad d_i \in \mathbb{F}_2. \quad (6)$$

Indeed, evaluating at $(0,0), (1,0), (0,1), (1,1)$ successively determines

$$d_0 = w(0,0), \quad d_1 = w(1,0) + d_0, \quad d_2 = w(0,1) + d_0, \quad d_3 = w(1,1) + d_0 + d_1 + d_2. \quad (7)$$

This standard representation [Car10] exposes the classification needed below: $d_3 = 0$ gives an *affine* update, while $d_3 = 1$ couples the retained state to the observed data. In the language of Post’s classification, the eight rules with $d_3 = 0$ constitute the affine class [Pos41]; no further lattice machinery is needed here.

rule	truth	ANF	kind	rule	truth	ANF	kind
W_0	0000	0	affine	W_8	1000	$1 + s + x + s x$	nonlinear
W_1	0001	$s x$	nonlinear	W_9	1001	$1 + s + x$	affine
W_2	0010	$s + s x$	nonlinear	W_{10}	1010	$1 + x$	affine
W_3	0011	s	affine	W_{11}	1011	$1 + x + s x$	nonlinear
W_4	0100	$x + s x$	nonlinear	W_{12}	1100	$1 + s$	affine
W_5	0101	x	affine	W_{13}	1101	$1 + s + s x$	nonlinear
W_6	0110	$s + x$	affine	W_{14}	1110	$1 + s x$	nonlinear
W_7	0111	$s + x + s x$	nonlinear	W_{15}	1111	1	affine

Table 2: The complete update-rule space. Addition and multiplication are in $\mathbb{F}_2$. “Nonlinear” means $d_3 = 1$ in ANF; the resulting recurrence has data-dependent state retention.

Two rules deserve names beyond their indices:

$$W_5(s, x) = x, \quad \text{last-observation update,} \quad (8)$$

$$W_6(s, x) = s + x, \quad \text{running-parity update.} \quad (9)$$

W_5 is the rule used by the earlier cxor implementation. It discards the old cell after every observation and stores the successor just seen. W_6 instead writes $s + x$, which is also the emitted residual r at that step. Along repeated visits to one context it accumulates the parity of all observed successors. Although the same emitter permits reading its state as a hard prediction, its update does not learn “what followed this context last” and is not naturally a next-symbol predictor. It enters because it is the algebraic counterpart of W_5 , not because it shares W_5 ’s modelling interpretation.

All sixteen rules use the same causal emitter and are therefore invertible by Theorem 2.1. The classification below asks what the update changes, rather than using invertibility to choose an update.

5 Structure of the transform

The address sequence interleaves 2^m one-cell machines. Once those machines are separated, the algebra of the sixteen rules becomes transparent.

5.1 Per-context decomposition

Fix an input and prefix. For a context a , let $k_{a,0} < k_{a,1} < \dots$ be its visits, set $y_{a,j} = x_{k_{a,j}}$, and let $u_{a,j}$ be cell a immediately before its j th visit. Directly from Algorithm 1,

$$r_{k_{a,j}} = y_{a,j} + u_{a,j}, \quad u_{a,j+1} = w(u_{a,j}, y_{a,j}). \quad (10)$$

No recurrence for a reads a cell belonging to another context. The full transform is therefore an interleaving of independent per-context recurrences; the source itself determines the interleaving through its preceding m -bit windows.

This decomposition also separates two roles that can otherwise be confused. Addressing decides *when* a state is consulted. The wiring decides how that state evolves *between visits to the same context*. Invertibility needs only that the address at the next step be recoverable from the decoded prefix, while the operator and adaptation properties below belong to the per-context recurrence.

5.2 Affine rules and three transfer operators

Assume first that $d_3 = 0$, so

$$w(s, x) = d_0 + d_1 s + d_2 x.$$

Within one context, let T mean “previous visit to this same context”: $(Tz)_0 = 0$ and $(Tz)_j = z_{j-1}$ for $j \geq 1$. The recurrence

$$u_{j+1} = d_0 + d_1 u_j + d_2 y_j, \quad r_j = y_j + u_j$$

gives, after collecting the terms that depend linearly on y ,

$$r = \frac{1 + (d_1 + d_2)T}{1 + d_1 T} y + c. \quad (11)$$

Here c is an additive response determined by the initial cell and d_0 . Division is only compact notation: if $d_1 = 1$, then

$$(1 + T)^{-1} = 1 + T + T^2 + \dots,$$

and only finitely many terms contribute at any visit. Equation (11) is the usual linear-recurrence calculation over $\mathbb{F}_2$ [Gol67; LC04], with delay measured in visits rather than adjacent tape positions.

Only (d_1, d_2) affects the linear operator, so the eight affine wirings realise exactly three:

(d_1, d_2)	operator	wirings	constant-free representative
$(0, 0)$ or $(1, 0)$	1	W_0, W_3, W_{12}, W_{15}	W_0, W_3
$(0, 1)$	$1 + T$	W_5, W_{10}	W_5
$(1, 1)$	$(1 + T)^{-1}$	W_6, W_9	W_6

Table 3: The affine update rules collapse to three per-context operators. Rules paired in a row differ only by an additive response.

For W_5 , the equation $r = (1+T)y+c$ is exactly the contextual difference in Equation (5). For W_6 , the state accumulates past symbols and the output is their running exclusive-or, giving $(1+T)^{-1}$. These are the only constant-free affine rules with nonidentity operators.

When $d_3 = 1$, the recurrence can instead be written

$$u_{j+1} = (d_0 + d_2 y_j) + (d_1 + y_j) u_j. \quad (12)$$

The multiplier of the old state now depends on the arriving symbol. The input-output map is in general nonlinear in y , so no fixed linear transfer operator represents these eight rules. They should be regarded as data-dependent updates, not as additional entries in Table 3.

The operator statement is per context, or equivalently conditional on a fixed address sequence. For $m \geq 2$, the partition of tape positions into contexts depends on the source, so T itself changes when the tape changes. The three-operator result must not be misread as claiming that the whole source-to-residual map is globally linear.

5.3 Adaptation and memory horizon

The ANF also says how long an old prediction can matter. Run the same context sequence twice with different initial cell bits, and let δ_j be the difference between the two states before visit j . Subtracting the two copies of Equation (6) gives

$$\delta_{j+1} = (d_1 + d_3 y_j) \delta_j, \quad \delta_j = \delta_0 \prod_{i < j} (d_1 + d_3 y_i). \quad (13)$$

Because the emitted residual contains the current state, the same δ_j measures the effect of the old cell on the output at visit j . Four adaptation regimes follow.

(d_1, d_3)	retention factor	wirings	adaptation horizon
(0, 0)	0	W_0, W_5, W_{10}, W_{15}	old state affects the first visit only
(1, 0)	1	W_3, W_6, W_9, W_{12}	old state is never forgotten
(0, 1)	y	W_1, W_4, W_{11}, W_{14}	retained through 1s; a 0 erases it
(1, 1)	$1 + y$	W_2, W_7, W_8, W_{13}	retained through 0s; a 1 erases it

Table 4: The four memory horizons. For the nonlinear rules, retention is controlled by the observed successor sequence within each context.

W_5 is immediately forgetting in the intended predictive sense: after one visit, its state is the new observation and contains no information about the previous belief. This is maximal adaptation for a one-bit hard predictor. W_6 is the opposite affine extreme: changing the initial cell changes every later state and residual in that context. The nonlinear rules offer conditional forgetting; whether an old state survives is decided by the observed symbol. Thus the affine/nonlinear split is also a split between fixed and data-dependent adaptation behaviour.

6 W_5 , W_6 , and the Gray-code special case

The two nontrivial affine operators describe very different state semantics. Along one context class with initial state q , Equations (8) and (9) unwind to

$$W_5 : \quad r_0 = y_0 + q, \quad r_j = y_j + y_{j-1} \quad (j \geq 1), \quad (14)$$

$$W_6 : \quad r_j = q + \sum_{i=0}^j y_i. \quad (15)$$

W_5 asks whether the current successor differs from the last successor of this context. Its state is a prediction with a clear learning rule. W_6 reports cumulative parity; because its new state equals the emitted r_j , it feeds the residual back into the next visit. Its state continues to contain the initial bit and all previous successors.

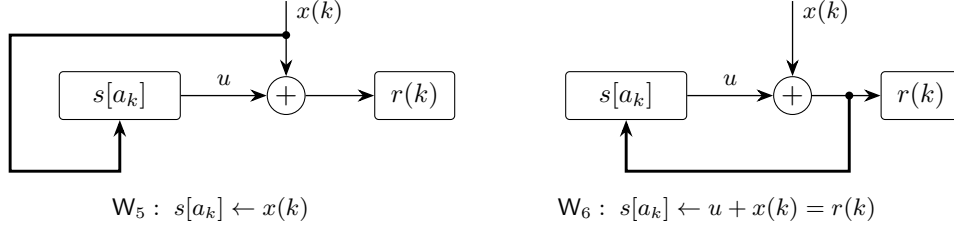


Figure 2: W_5 and W_6 use the same emitter. Moving the feedback tap from before the exclusive-or to after it changes a last-observation predictor into a running-parity state.

The operators $1 + T$ and $(1 + T)^{-1}$ are mutual inverses, but this statement has a precise scope. If the address sequence is held fixed, applying the W_6 recurrence to the W_5 residual (or vice versa) restores each per-context sequence. In *cxor*, however, addresses are generated from the transform’s current input. Feeding a W_5 residual into a fresh W_6 encoder generally produces a different address sequence and is not the decoder of Section 2. The causal decoder reconstructs source bits and reproduces the original addresses while applying the *same* update rule. Operator duality and transform inversion are therefore related but distinct facts.

6.1 Order zero

At $m = 0$ there is only one context, so visits are ordinary adjacent positions and T is the usual one-step delay. With a zero initial cell, W_5 gives

$$r_0 = x_0, \quad r_k = x_k + x_{k-1} \quad (k \geq 1). \quad (16)$$

If the bits are read most significant first as the binary representation of an integer b , Equation (16) is exactly reflected binary (Gray) encoding

$$g = b \oplus (b \gg 1)$$

[Gra53]. W_6 takes prefix exclusive-ors and is the corresponding Gray decoder. This is the simplest concrete instance of the two transfer operators.

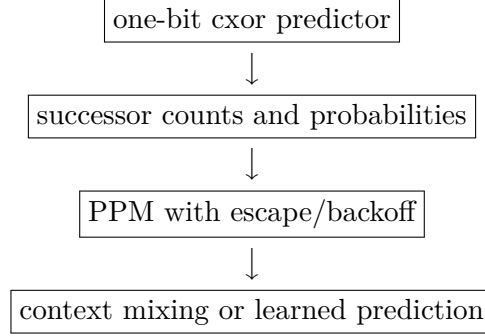
For example, 90 is 01011010 in eight-bit binary. W_5 emits 01110111, representing $119 = 90 \oplus 45$, and W_6 maps that word back to 01011010. In this order-zero case the fixed-address operator duality is also an ordinary composition of the two transforms, because there is no source-dependent address sequence to change between passes.

For $m > 0$, the delay means “previous visit to this context” rather than “previous tape position.” At $m = 1$ with zero table and prefix, W_5 happens to reduce to the stride-two difference $r_k = x_k + x_{k-2}$, taking out-of-range bits as zero. At larger orders the visit partition is source-dependent and no fixed tape stride describes it. Calling *cxor* a generalised Gray code would therefore hide its defining feature: context-addressed prediction. Gray coding is an elegant boundary case, not the main interpretation.

7 From cxor back to statistical compression

cxor_{W_5} can be read as a context model reduced to one bit, and its limitations are exactly what that reduction removes. A cell can say “zero” or “one,” but not “one with probability 0.9.” It cannot distinguish a prediction supported by one observation from one supported by a thousand. W_5 changes its mind after every error, has no escape when a context is new, and cannot share evidence between related contexts.

Putting capacity back gives a useful conceptual ladder:



The first step replaces a hard remembered successor by evidence about both possible successors. PPM combines context orders through an explicit escape mechanism [BCW90]. Context mixing combines predictions from many models, often with adaptive weights [Mah05]. These are not variants among the sixteen wirings: widening the state leaves the finite one-bit design space.

What survives is the causal architecture. At position k , a deterministic predictor whose state is reproducible from $x_0, \dots, x_{k-1}$ may produce a hard decision $\hat{x}_k$, and

$$r_k = x_k + \hat{x}_k$$

remains an invertible residual for exactly the reason used in Theorem 2.1. The predictor may contain counts, mixtures, or learned state; the decoder recovers $x_k = r_k + \hat{x}_k$ and then performs the same state update. Exact reproducibility, including numerical arithmetic, is essential.

Probabilistic compressors normally do something more informative. They pass the probability $P(x_k = 1 \mid x_{<k})$ to an arithmetic or range coder rather than threshold it to one bit and discard confidence. cxor_{W_5} isolates the degenerate hard-prediction case, in which correctness alone is represented. It is useful analytically because its state and complete update space are small, not because one bit is expected to compete with a statistical model.

The practical question left by the example in Section 1 is correspondingly narrow: for which sources, context orders, and downstream coders does contextual differencing expose structure that the back end models more cheaply than the original sequence? Answering it needs comparative experiments that include the model description, initialisation, coder, and any side information. Zero enrichment alone is a diagnostic of prediction success, not a compression result.

8 Conclusion

cxor is a particularly small member of the broader CTA family: each binary m -bit context addresses one bit of state, and an XOR emitter combines that state with the source.

Under the historically implemented W_5 update, the state is a hard prediction equal to the successor most recently observed for that context. The transform can therefore suppress stable context-to-successor associations, producing a zero-rich residual on favourable inputs. It remains a length-preserving bijection, not a compressor; any coding benefit must come from a downstream model and coder.

Varying the one-bit update exposes a complete sixteen-rule space within this architecture. Causal sequential decoding works for every rule. Within each interleaved context class, the eight affine rules collapse to the operators 1 , $1 + T$, and $(1 + T)^{-1}$, while the eight nonlinear rules use an observation-dependent retention factor. The same factor gives four adaptation horizons. W_5 is the immediately adapting last-observation predictor; W_6 is its running-parity operator counterpart and never forgets its initial state. Gray coding appears cleanly at $m = 0$, but context-addressed prediction is the general cxor construction.

Moving upward from one bit to counts, probabilities, PPM, and context mixing sacrifices this tiny classification but preserves the central principle: a causal predictor can be paired with a reversible residual. In this sense, cxor provides an analytically complete base case for richer predictive models.

Code availability. The sources of this paper, the reference implementation of Appendix A, the historical `cta.py`, and the verification harnesses of Appendix C are available in the accompanying repository [Yak26] at <https://github.com/ewiger/cxor-paper>.

A Reference implementation

The following dependency-free code implements the sixteen-rule transform and its causal decoder. The wiring index uses the truth-table convention of Section 4; selecting $k=5$ gives the historical cxor instance. This listing, the historical implementation, and the harnesses of Appendix C are all contained in the paper’s repository [Yak26].

```
def wiring(k, s, x):
    """W_k(s,x), where k's bits are w(0,0),w(0,1),w(1,0),w(1,1)."""
    row = 2 * s + x
    return (k >> (3 - row)) & 1

def encode(bits, m=4, k=5, table=None, prefix=None):
    table = list(table) if table is not None else [0] * (1 << m)
    context = list(prefix) if prefix is not None else [0] * m
    residual = []
    for x in bits:
        a = int("".join(map(str, context)), 2) if m else 0
        s = table[a]
        residual.append(x ^ s)
        table[a] = wiring(k, s, x)
        if m:
            context = context[1:] + [x]
    return residual

def decode(residual, m=4, k=5, table=None, prefix=None):
    table = list(table) if table is not None else [0] * (1 << m)
    context = list(prefix) if prefix is not None else [0] * m
    bits = []
    for r in residual:
```

```

    a = int("".join(map(str, context)), 2) if m else 0
    s = table[a]
    x = r ^ s
    bits.append(x)
    table[a] = wiring(k, s, x)
    if m:
        context = context[1:] + [x]
    return bits

```

The historical `cta.py` of [Yak26] uses a `bitarray` table and performs the W_5 assignment only when the residual is one. As shown in Section 2, omitting the assignment when the residual is zero leaves exactly the same state. Its inverse reconstructs $x = r \oplus s[a]$, writes x to the cell, and shifts the recovered bit into the context, matching `decode(..., k=5)` above.

B Further structure of the one-bit model

The one-bit model has two further structural properties: affine dependence on the initial table, and a delay before an update rule becomes observable in the output.

B.1 Affine dependence on the initial table

The split in Section 5 concerns linearity in the *successor sequence*. With the source held fixed, a different fact holds: the transform is affine in its initial table for all sixteen wirings, including those with $d_3 = 1$.

To see this, factor the ANF as

$$w(s, x) = \underbrace{(d_0 + d_2 x)}_{\alpha(x)} + \underbrace{(d_1 + d_3 x)}_{\beta(x)} s. \quad (17)$$

Fix one context with successor sequence $y_0, y_1, \dots$ and initial cell q . Its recurrence has the form

$$u_{j+1} = \alpha(y_j) + \beta(y_j)u_j.$$

Induction gives

$$u_j = \mu_j + \lambda_j q, \quad \lambda_j = \prod_{i < j} \beta(y_i), \quad (18)$$

where μ_j and λ_j depend on the fixed source but not on q . Since $r_{k_j} = y_j + u_j$, every residual in that context is affine in its own initial cell.

Cells never interact, and the address sequence depends on the source and prefix rather than the initial table. Consequently, for fixed (π, x) there is a linear map $P_{\pi, x} : \mathbb{F}_2^{2^m} \rightarrow \mathbb{F}_2^n$ such that

$$\text{cxor}_w(q, \pi, x) = \text{cxor}_w(0, \pi, x) + P_{\pi, x} q \quad (19)$$

for every wiring w . This does not contradict the nonlinear classification: when $d_3 = 1$, $\beta(x)$ is a source-dependent coefficient, which is constant only because x is held fixed in Equation (19).

The columns of $P_{\pi, x}$ have disjoint supports: changing cell a can affect only visits to context a . Moreover, a visited cell always affects its first visit, because the empty product in Equation (18) is $\lambda_0 = 1$. It follows immediately that

$$\text{rank } P_{\pi, x} = |\{a : a \text{ is visited by } (\pi, x)\}| \leq 2^m. \quad (20)$$

Thus a known source/residual pair determines the initial bits of all visited cells, while unvisited cells are necessarily unobservable. Equation (13) refines the same calculation by describing the full support of each nonzero column.

B.2 When does the wiring become visible?

At a context’s first visit, the emitted residual is $y_0 + q$ and is independent of the update rule; the rule acts only after that bit has been emitted. Differences between two wirings can therefore be observed only on a later visit to a cell on which their earlier updates disagreed.

W_5 and W_6 have an additional one-visit agreement when the table starts at zero. On the first visit both store y_0 . They consequently emit the same residual on the second visit, but then store respectively

$$W_5(y_0, y_1) = y_1, \quad W_6(y_0, y_1) = y_0 + y_1.$$

If $y_0 = 1$, that difference becomes observable on the third visit to the context. For a zero prefix, the tape 10^{m+2} supplies such a trace in the all-zero context and separates the two rules at its last symbol; its length is $m + 3$. The verification harness confirms that no shorter tape separates them for $m \leq 6$. Thus the W_5/W_6 distinction is invisible until some context has accumulated enough visit history.

C Computational checks

The proofs in the body do not depend on experiments. A deterministic, dependency-free harness in `cta_spec.py` [Yak26] nevertheless checks the implementation against the main claims. Running `make check` performs, among other tests:

- exhaustive truth-table indexing and ANF classification for all sixteen wirings;
- causal-prefix checks for all rules, exhaustive bijectivity on short tapes, and round trips through the sequential decoder for all rules and all tables and prefixes in the tested ranges;
- the transfer formula for all eight affine rules and both initial cell values, together with direct nonlinearity tests for the eight $d_3 = 1$ rules;
- the four state-retention regimes of Equation (13), checked position-by-position for all sixteen rules; and
- exhaustive order-zero Gray conversion over all 2^{12} twelve-bit words, plus the stride-two identity at order one through length 13.

The separate `cta_w5_w6.py` harness of the same repository reproduces the reference 33-bit example, compares the W_5 and W_6 interpretations, exercises the causal inverse, and verifies their mutual inverse relationship when the address sequence is frozen. The earlier implementation’s output agrees with W_5 :

101001110111000011100001000000001.

The generated checks are corroboration and regression tests; causality, the per-context operator classification, and the adaptation horizons are established algebraically in the paper.

References

- [BCW90] Timothy C. Bell, John G. Cleary, and Ian H. Witten. *Text Compression*. Prentice Hall, 1990.
- [BW94] Michael Burrows and David J. Wheeler. *A Block-sorting Lossless Data Compression Algorithm*. Tech. rep. 124. Digital Equipment Corporation, Systems Research Center, 1994.
- [Car10] Claude Carlet. “Boolean Functions for Cryptography and Error Correcting Codes”. In: *Boolean Models and Methods in Mathematics, Computer Science, and Engineering*. Ed. by Yves Crama and Peter L. Hammer. Cambridge University Press, 2010, pp. 257–397.
- [Gol67] Solomon W. Golomb. *Shift Register Sequences*. Holden-Day, 1967.
- [Gra53] Frank Gray. *Pulse Code Communication*. U.S. Patent 2,632,058, filed 13 November 1947, issued 17 March 1953. 1953.
- [LC04] Shu Lin and Daniel J. Costello. *Error Control Coding*. 2nd ed. Prentice Hall, 2004.
- [Mah05] Matthew V. Mahoney. “Adaptive weighing of context models for lossless data compression”. In: 2005.
- [Pos41] Emil L. Post. *The Two-Valued Iterative Systems of Mathematical Logic*. Annals of Mathematics Studies 5. Princeton University Press, 1941.
- [Yak26] Yauhen Yakimovich. *cxor: paper sources, reference implementation, and verification harness*. Git repository. 2026. URL: <https://github.com/ewiger/cxor-paper> (visited on 08/20/2026).